

Section 5 — Custom Validators

Concept Notes

Sometimes basic types and field constraints are not enough. You may need your own validation logic, such as checking that a username does not contain spaces or that a discount price is lower than the original price.

Pydantic provides **custom validators** for writing such validation rules.

Field Validator

A field validator checks one specific field.

For example, it can check whether:

- A username contains spaces
- A password is strong enough
- An age follows a custom rule
- A name contains valid characters

In Pydantic V2, `@field_validator` is used for field-level validation.

Model Validator

A model validator checks multiple fields or the complete model together.

It is useful when one field depends on another field.

For example:

- Password and confirm password must match
- End date must be after start date
- Discount price must be lower than original price

In Pydantic V2, `@model_validator` is used for model-level validation.

Before Validation

A validator can run before normal Pydantic validation.

It receives the raw incoming data before standard type conversion and validation are completed.

This is useful for cleaning or transforming raw input.

After Validation

A validator can run after normal Pydantic validation.

At this stage, values have already passed standard validation and conversion.

This is useful for checking business rules using validated values.

Important Points

- Custom validators are used for application-specific validation rules.
- `@field_validator` validates individual fields.
- `@model_validator` validates the model as a whole.
- Field validators are useful for rules related to one field.
- Model validators are useful for rules involving multiple fields.
- `mode="before"` works with raw incoming data.
- `mode="after"` works after standard validation.
- A validator can raise `ValueError` when validation fails.
- Validators help keep validation logic inside the data model.

Syntax

Field Validator

```
@field_validator("field_name")
```

Create validation method

↓

Receive field value



Check custom rule



Invalid → Raise ValueError

Valid → Return value

Model Validator

@model_validator(mode="after")

Receive validated model



Compare multiple fields



Invalid → Raise ValueError

Valid → Return model

Validation Timing

mode="before"



Raw Input



Standard Pydantic Validation

Standard Pydantic Validation



mode="after"



Final Business Rule Check